40-background-jobs

40. Background Jobs & Queue

Level: Intermediate
Jam: 8 (4 sesi)
Prasyarat: Paham Node.js, Express, Redis dasar
Output: Queue-powered app — email notification system

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Paham masalah sync vs async processing dan kenapa queue dibutuhin
- Ngerti konsep queue: producer, consumer, broker, job
- Setup BullMQ + Redis buat job processing
- Handle job scheduling, retry, concurrency, dan progress reporting
- Implementasi use case nyata: email, PDF, image processing
- Deploy queue system production-ready dengan dashboard & monitoring

Materi

Sesi	Topik	File
1	Queue Concepts — sync vs async, Redis, BullMQ setup, job lifecycle	01-queue-concepts.md
2	BullMQ Advanced — scheduling, concurrency, retry, events, sandbox	02-bullmq-advanced.md

Sesi	Topik	File
3	Real World Queue — email, PDF, image, notification batching, bulk export	03-real-world-queue.md
4	Queue Production — Bull Board, dead letter, scaling, graceful shutdown	04-queue-production.md

Output Akhir Modul

> **Queue-Powered Notification System** — aplikasi yang antre dan proses task async pake BullMQ + Redis. Bisa kirim email, generate PDF report, resize image, dan monitoring via dashboard. Semua job punya retry logic, progress tracking, dan graceful shutdown.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Compare synchronous vs asynchronous processing with sequence diagrams"
- "Explain how Redis pub/sub and list data structures can implement a job queue"
- "Generate BullMQ code for a job that sends welcome email with retry logic of 3 attempts and exponential backoff"
- "What happens to incomplete jobs if my Node worker crashes? How do I recover them?"
- "Design a queue architecture for a platform that needs PDF invoice generation, image thumbnails, and batch email — including error handling and scaling strategy"

01. Queue Concepts

Sync vs Async Problem

Synchronous (Blocking)

Tanpa queue, semuanya jalan berurutan. Satu request lambat bisa nge-blok yang lain.

```
sequenceDiagram
    participant Client
    participant Server
    participant Email
    Client->>Server: POST /register
    Server->>Server: Simpan user ke DB
    Server->>Email: Kirim welcome email (3 detik)
    Email-->>Server: Email terkirim
    Server-->>Client: Response 201 (setelah 3+ detik)
    Note over Client,Server: Client nunggu 3+ detik! 
```

Masalah: user experience jelek, server gak scalable, request lain ngantre.

Asynchronous (Queue)

Queue pisahin request dari proses berat. Response langsung balik, task jalan di background.

```
sequenceDiagram
    participant Client
    participant Server
    participant Redis
    participant Worker
    participant Email
    Client->>Server: POST /register
    Server->>Server: Simpan user ke DB
    Server->>Redis: QUEUE email.welcome (job data)
    Server-->>Client: Response 201 (50ms! )
    Worker->>Redis: BLRPOP email.welcome
    Redis-->>Worker: Job data
    Worker->>Email: Kirim email (3 detik)
    Note over Worker,Email: User udah dapet response
```

Keuntungan: - Response cepet — user gak nunggu - Worker bisa scale horizontal - Job bisa retry kalau gagal - Anti kehilangan task (persistent di Redis)

Queue Concepts

Komponen Utama

Istilah	Deskripsi	Analogi Kasir
Queue	Struktur data FIFO tempat job antre	Antrian pembayaran
Producer	Kode yang nambahin job ke queue	Pelanggan ambil nomor antrean
Consumer/Worker	Kode yang ambil dan proses job	Kasir yang layani pelanggan
Broker	Middleware yang nyimpen dan distribusi job	Sistem nomor antrean
Job	Unit kerja yang diproses (data + metadata)	Satu transaksi belanja
Payload	Data yang dibawa job (JSON)	Isi keranjang belanja

Producer → Queue → Consumer Flow

```
graph LR
    P1[Producer: API] --> Q[(Queue)]
    P2[Producer: Cron] --> Q
    Q --> C1[Worker 1]
    Q --> C2[Worker 2]
    Q --> C3[Worker 3]
    style Q fill:#fff3e0,stroke:#e65100
    style C1 fill:#e8f5e9,stroke:#2e7d32
    style C2 fill:#e8f5e9,stroke:#2e7d32
    style C3 fill:#e8f5e9,stroke:#2e7d32
```

Queue bersifat **FIFO** (First In, First Out) — job pertama masuk, pertama diproses.

Redis Overview

Redis = **Re**mote **Di**ctionary **S**erver. In-memory data store super cepat, sering dipakai buat queue.

Kenapa Redis Cocok buat Queue?

1. **In-memory** — operasi sub-millisecond
2. **Persistent** — bisa disimpen ke disk (RDB/AOF)
3. **Pub/Sub** — realtime messaging

4. **List** — natural buat FIFO queue (LPUSH/BRPOP)
5. **Stream** — queue canggih dengan consumer groups (Redis 5+)
6. **Sorted Set** — buat delayed job / scheduling

Data Structure untuk Queue

```
# Simple queue pake List
LPUSH myqueue "job1"       # Tambah job
LPUSH myqueue "job2"
BRPOP myqueue 0            # Ambil job (blocking)

# Delayed job pake Sorted Set
ZADD delayed:email 1730000000 "job:abc123"
ZRANGEBYSCORE delayed:email -inf 1730000000  # Ambil yg udah waktunya

# Pub/Sub untuk notifikasi
PUBLISH channel:job:completed "job:abc123"
SUBSCRIBE channel:job:completed
```

Setup Redis

```
# Install Redis (Ubuntu/Debian)
sudo apt-get install redis-server -y

# Start Redis
sudo systemctl start redis-server

# Cek koneksi
redis-cli ping
# Output: PONG

# Atau pake Docker
docker run -d --name redis -p 6379:6379 redis:7-alpine
```

BullMQ Setup + First Queue

Apa itu BullMQ?

BullMQ = library Node.js buat job queue berbasis Redis. Dibuat oleh OptimalBits, mature dan production-ready.

Instalasi

```
mkdir queue-demo && cd queue-demo
npm init -y
npm install bullmq ioredis
```

Koneksi Redis

```
// connection.js
const IORedis = require('ioredis');

const connection = new IORedis({
  host: 'localhost',
  port: 6379,
  maxRetriesPerRequest: null, // penting buat BullMQ
});

module.exports = connection;
```

Buat Queue (Producer)

```
// producer.js
const { Queue } = require('bullmq');
const connection = require('./connection');

const emailQueue = new Queue('email', { connection });

async function addJobs() {
  // Tambah job ke queue
  const job = await emailQueue.add('welcome-email', {
    to: 'user@example.com',
    name: 'Budi',
    template: 'welcome',
  });

  console.log(`Job ${job.id} ditambahkan ke queue`);
  await connection.quit();
}

addJobs();
```

Buat Worker (Consumer)

```
// worker.js
const { Worker } = require('bullmq');
```

```
const connection = require('./connection');

const worker = new Worker('email', async (job) => {
  console.log(`Processing job ${job.id}...`);
  console.log(`Data:`, job.data);

  // Simulasi proses (kirim email)
  await new Promise((resolve) => setTimeout(resolve, 2000));

  console.log(`Job ${job.id} selesai!`);
  return { sent: true, to: job.data.to };
}, { connection });

worker.on('completed', (job) => {
  console.log(` ${job.id} completed`);
});

worker.on('failed', (job, err) => {
  console.log(` ${job.id} failed:`, err.message);
});

console.log('Worker started...');
```

Jalankan

```
# Terminal 1 — start worker
node worker.js

# Terminal 2 — tambah job
node producer.js
```

Output:

```
# Worker
Worker started...
Processing job 1...
Data: { to: 'user@example.com', name: 'Budi', template: 'welcome' }
Job 1 selesai!
 1 completed
```

Job Lifecycle

BullMQ job melewati state berikut:

```
stateDiagram-v2
    [*] --> Waiting: add()
    Waiting --> Active: Worker ambil
    Active --> Completed: Sukses
    Active --> Failed: Error
    Active --> Waiting: Retry (attempts left)
    Waiting --> Delayed: Scheduling
    Delayed --> Waiting: Waktunya tiba
    Failed --> Waiting: Manual retry
    Failed --> [*]: Max attempts reached
    Completed --> [*]
```

State	Deskripsi
Waiting	Job antre, belum diproses
Active	Lagi dikerjain worker
Completed	Berhasil, return value disimpan
Failed	Gagal (error), bisa retry
Delayed	Ditunda sampai waktu tertentu
Paused	Queue di-pause, gak ada job diproses

Cek Status Job

```js
// status-check.js
const { Queue } = require('bullmq');
const connection = require('./connection');

async function checkQueue() {
  const queue = new Queue('email', { connection });

  const waiting = await queue.getWaiting();
  const active = await queue.getActive();
  const completed = await queue.getCompleted();
  const failed = await queue.getFailed();
  const delayed = await queue.getDelayed();

  console.log({
    waiting: waiting.length,
    active: active.length,
    completed: completed.length,
    failed: failed.length,
    delayed: delayed.length,
  });

  await connection.quit();
```

```
}

checkQueue();
```

Latihan

Latihan 1: Setup Redis + BullMQ

Setup Redis (local atau Docker). Buat project Node.js dengan BullMQ. Buat queue `notifications` dengan 1 worker dummy yang console.log data job.

Latihan 2: Queue dengan Repeatable Job

Buat producer yang nambah 5 job ke queue `file-processing`. Worker simulasi proses file (setTimeout 1 detik per job). Jalankan semua job dan pastikan selesai.

Latihan 3: Job Lifecycle Observer

Buat worker yang process job dengan 50% chance error. Register event listener `completed`, `failed`, `progress`. Catat jumlah completed vs failed.

Latihan 4: Delayed Job

Buat producer yang nambah job dengan `delay: 5000` (5 detik). Worker console.log timestamp saat job mulai diproses. Buktikan job delay 5 detik dari waktu add.

02. BullMQ Advanced

Job Scheduling

Cron / Repeatable Jobs

BullMQ bisa jadwalin job berulang otomatis pake cron expression.

```
// schedule.js
const { Queue } = require('bullmq');
const connection = require('./connection');

const reportQueue = new Queue('reports', { connection });
```

```
async function setupSchedules() {
  // Setiap jam (menit 0)
  await reportQueue.add('hourly-summary', { type: 'hourly' }, {
    repeat: { pattern: '0 * * * *' },
  });

  // Setiap hari jam 08:00
  await reportQueue.add('daily-report', { type: 'daily' }, {
    repeat: { pattern: '0 8 * * *' },
    jobId: 'daily-report', // ID tetap biar gak dobel
  });

  // Setiap Senin jam 09:00
  await reportQueue.add('weekly-digest', { type: 'weekly' }, {
    repeat: { pattern: '0 9 * * 1' },
  });

  console.log('Schedules created');
  await connection.quit();
}

setupSchedules();
```

```
// worker-schedule.js
const { Worker } = require('bullmq');
const connection = require('./connection');

const worker = new Worker('reports', async (job) => {
  console.log(`[${new Date().toISOString()}] Running ${job.name}`);
  console.log(`Type: ${job.data.type}`);

  switch (job.data.type) {
    case 'hourly':
      // Aggregasi data per jam
      break;
    case 'daily':
      // Generate report harian
      break;
    case 'weekly':
      // Kirim digest mingguan
      break;
  }
}, { connection });

console.log('Schedule worker started...');
```

Delay Job

Job bisa ditunda tanpa cron — sekali jalan aja setelah delay.

```
// Kirim email promo besok
await emailQueue.add('promo-email', {
  userId: '123',
  promo: 'DISKON50',
}, {
  delay: 24 * 60 * 60 * 1000, // 1 hari dalam ms
});

// Kirim reminder 30 menit sebelum event
await reminderQueue.add('event-reminder', {
  eventId: 'evt_001',
  startAt: '2025-07-05T14:00:00Z',
}, {
  delay: 30 * 60 * 1000,
});
```

Buat Cron Jobs dari String

Bantuan cron expression:

```
* * * * *          Setiap menit
*/5 * * * *        Setiap 5 menit
0 * * * *          Setiap jam (menit 0)
0 8 * * *          Setiap hari jam 08:00
0 9 * * 1          Setiap Senin jam 09:00
0 0 1 * *          Tanggal 1 setiap bulan
0 0 * * 0          Setiap Minggu tengah malam
```

Job Concurrency & Rate Limit

Concurrency (Parallel Processing)

Worker bisa proses banyak job sekaligus.

```
// Worker dengan 5 concurrent jobs
const worker = new Worker('email', async (job) => {
  await sendEmail(job.data);
}, {
  connection,
  concurrency: 5, // proses 5 job barengan
});
```

Tanpa concurrency, worker cuma proses 1 job per kali.

```
graph TD
    subgraph "Concurrency=1 (Sequential)"
        J1[Job 1] --> J2[Job 2] --> J3[Job 3]
    end
    subgraph "Concurrency=3 (Parallel)"
        A1[Job A] --> A1d[Done]
        A2[Job B] --> A2d[Done]
        A3[Job C] --> A3d[Done]
    end
```

Rate Limit

Batasi jumlah job per periode waktu — penting buat API eksternal.

```
const worker = new Worker('email', async (job) => {
  await sendEmailViaAPI(job.data);
}, {
  connection,
  limiter: {
    max: 10,       // maksimal 10 job
    duration: 1000, // per 1 detik
  },
});
```

Ini berguna kalau: - API email cuma allow 10 request/detik - Database gak sanggup kena 100 query sekaligus - Hindari rate limit dari service eksternal

Multiple Workers

Jalanin banyak worker instance di process terpisah buat scale.

```
# Terminal 1
node worker.js

# Terminal 2
node worker.js

# Terminal 3 — producer
node producer.js
```

Masing-masing worker ambil job dari queue yang sama. Redis handle distribusi job antar worker.

Retry Strategy

Auto Retry

Worker bisa otomatis retry job yang gagal.

```
// worker-retry.js
const { Worker } = require('bullmq');
const connection = require('./connection');

const worker = new Worker('email', async (job) => {
  // Simulasi error 30% chance
  if (Math.random() < 0.3) {
    throw new Error('Gagal kirim email');
  }
  console.log(`□ Email ke ${job.data.to} terkirim`);
}, {
  connection,
  concurrency: 3,
});

console.log('Retry worker started...');
```

Konfigurasi Retry

```
// producer dengan retry options
const { Queue } = require('bullmq');

const emailQueue = new Queue('email', { connection });

async function addWithRetry() {
  await emailQueue.add('welcome-email', {
    to: 'user@example.com',
  }, {
    attempts: 5,               // max 5x percobaan
    backoff: {
      type: 'exponential',     // exponential backoff
      delay: 2000,             // delay awal 2 detik
    },
  });

  await emailQueue.add('invoice', {
    to: 'client@company.com',
  }, {
    attempts: 3,
    backoff: {
      type: 'fixed',           // delay tetap
```

```
      delay: 5000,              // 5 detik antar percobaan
    },
  });
}
```

Backoff Strategy

Type	Behavior	Contoh
fixed	Delay sama tiap retry	5s, 5s, 5s
exponential	Delay nambah eksponensial	2s, 4s, 8s, 16s

Perhitungan exponential: `delay * 2^(attempt - 1)`

Attempt 1: 2000ms (gagal) Attempt 2: 4000ms (gagal) Attempt 3: 8000ms (gagal) Attempt 4: 16000ms (gagal) Attempt 5: 32000ms — last attempt

Custom Retry Logic

Kadang kita cuma mau retry untuk error tertentu.

```
const worker = new Worker('email', async (job) => {
  try {
    await sendEmail(job.data);
  } catch (err) {
    if (err.code === 'RATE_LIMITED') {
      // Rate limit — pantas retry
      throw err;
    }
    if (err.code === 'INVALID_EMAIL') {
      // Email salah — guna retry
      await job.discard(); // Tandai selesai tanpa retry
      return { skipped: true, reason: 'Invalid email' };
    }
    // Error lain — retry
    throw err;
  }
}, { connection });
```

Job Events & Progress Reporting

Events yang Tersedia

```
const worker = new Worker('email', async (job) => {
  await job.updateProgress(10); // progress 10%
  // proses...
  await job.updateProgress(50);
  // proses...
  await job.updateProgress(100);
  return { success: true };
}, { connection });

// Event listeners
worker.on('completed', (job, returnvalue) => {
  console.log(`✅ ${job.id} selesai:`, returnvalue);
});

worker.on('failed', (job, err) => {
  console.log(`❌ ${job.id} gagal:`, err.message);
});

worker.on('progress', (job, progress) => {
  console.log(`📊 ${job.id} progress: ${progress}%`);
});

worker.on('active', (job) => {
  console.log(`🔄 ${job.id} mulai diproses`);
});

worker.on('stalled', (job) => {
  console.log(`⚠️ ${job.id} stalled — worker crash?`);
});

// Queue-level events
const queue = new Queue('email', { connection });
queue.on('waiting', (job) => {
  console.log(`⏳ ${job.id} masuk antrian`);
});

queue.on('paused', () => console.log('⏸ Queue paused'));
queue.on('resumed', () => console.log('▶ Queue resumed'));
```

Progress Reporting Detail

Buat job panjang, progress kasih feedback ke user.

```
// bulk-import.js
const { Worker } = require('bullmq');

const worker = new Worker('import', async (job) => {
  const { users } = job.data; // array of 1000 users
  const total = users.length;

  for (let i = 0; i < total; i++) {
    await saveUser(users[i]);

    // Update progress tiap 10%
    const percent = Math.round(((i + 1) / total) * 100);
    if (percent % 10 === 0) {
      await job.updateProgress(percent);
      console.log(`Progress: ${percent}% (${i + 1}/${total})`);
    }
  }

  return { imported: total };
}, { connection });

// Client bisa cek progress via API
// GET /api/jobs/:id/progress
```

Sandbox Processors

Masalah Worker Biasa

Worker jalan di process yang sama. Kalau worker ada memory leak atau crash, process server ikut mati.

```
// □ Berbahaya — worker dan server satu process
const worker = new Worker('heavy', async (job) => {
  // Proses berat yg bisa crash
  heavyComputation(job.data); // crash? server ikut mati!
}, { connection });
```

Sandbox Processor

Jalanin tiap job di process terpisah (child process). Aman — kalau job crash, worker utama tetep jalan.

```
// worker-sandbox.js
const { Worker } = require('bullmq');
const path = require('path');
```

```
const connection = require('./connection');

const worker = new Worker('image', path.join(__dirname, 'sandbox.js'), {
  connection,
  concurrency: 3,
});

worker.on('completed', (job) => {
  console.log(`□ ${job.id} selesai`);
});

worker.on('failed', (job, err) => {
  console.log(`□ ${job.id} gagal:`, err.message);
});
```

```
// sandbox.js — file terpisah dijalankan sebagai child process
module.exports = async (job) => {
  const { imagePath, effect } = job.data;

  console.log(`[Sandbox ${process.pid}] Proses ${imagePath}`);

  // Kalau crash, cuma child process yang mati
  const result = await processImage(imagePath, effect);

  return result;
};
```

Sandbox cocok buat: - Heavy computation (image processing, PDF) - Kode yang gak stabil / rawan crash - Isolasi memory — memory leak gak ngaruh ke main process - Hot reload development

---

Latihan

Latihan 1: Cron Scheduler

Buat queue `cleanup` dengan worker yang jalan setiap jam (pattern `0 * * * *`). Worker log "Cleanup running at [timestamp]". Biarkan jalan 3 menit, amati pola eksekusi.

Latihan 2: Retry + Backoff

Buat queue `unstable-service` dengan worker yang throw error random 70% chance. Tambah job dengan `attempts: 5` dan `backoff.exponential: 1000`. Catat jumlah attempts tiap job dari event listener `failed`.

Latihan 3: Progress Report (Heavy Task)

Buat queue `bulk-export`. Worker simulasi export 100 records — update progress tiap 10 record. Producer nambah 1 job, lalu polling progress via `job.getProgress()` setiap 1 detik sampai 100%.

Latihan 4: Sandbox Processor

Buat queue `dangerous-job`. Worker pake sandbox processor (file terpisah). Di sandbox, simulasi error dengan `throw new Error('Simulated crash')`. Pastikan worker utama TIDAK crash. Job marked as failed, worker tetep jalan buat job berikutnya.

03. Real World Queue

1. Email Sending (Nodemailer + Queue)

Kirim email lewat API itu lambat dan gak bisa diandelin di request cycle. Queue jadi solusi.

Setup Nodemailer

```
npm install nodemailer bullmq ioredis
```

Producer — Queue Email

```
// email-producer.js
const { Queue } = require('bullmq');
const connection = require('./connection');

const emailQueue = new Queue('email', { connection });

async function sendWelcomeEmail(user) {
  const job = await emailQueue.add('welcome-email', {
    to: user.email,
    subject: 'Selamat datang di Platform!',
    template: 'welcome',
    data: {
      name: user.name,
      verifyLink: `https://app.com/verify/${user.token}`,
    },
  });
  console.log(`Email job ${job.id} untuk ${user.email}`);
  return job;
```

```
}

async function sendResetPassword(email, resetToken) {
  await emailQueue.add('reset-password', {
    to: email,
    subject: 'Reset Password',
    template: 'reset',
    data: { resetToken },
  }, {
    attempts: 3,
    backoff: { type: 'exponential', delay: 2000 },
  });
}

module.exports = { sendWelcomeEmail, sendResetPassword };
```

Worker — Kirim Email via Nodemailer

```
// email-worker.js
const { Worker } = require('bullmq');
const nodemailer = require('nodemailer');
const connection = require('./connection');

// Konfigurasi transporter
const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST || 'smtp.mailtrap.io',
  port: process.env.SMTP_PORT || 587,
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASS,
  },
});

const templates = {
  'welcome': (data) => ({
    html: `<h1>Halo ${data.name}!</h1><p>Selamat datang. Verifikasi: <a href="${
  }),
  'reset': (data) => ({
    html: `<h1>Reset Password</h1><p>Token: ${data.resetToken}</p>`,
  }),
};

const worker = new Worker('email', async (job) => {
  await job.updateProgress(10);
```

```
  const template = templates[job.data.template];
  if (!template) throw new Error(`Template ${job.data.template} gak ditemukan`);

  const mailOptions = {
    from: '"Platform" <noreply@platform.com>',
    to: job.data.to,
    subject: job.data.subject,
    ...template(job.data.data),
  };

  await job.updateProgress(50);

  // Kirim email
  const info = await transporter.sendMail(mailOptions);

  await job.updateProgress(100);
  return { messageId: info.messageId, accepted: info.accepted };

}, { connection, concurrency: 5 });

worker.on('completed', (job, result) => {
  console.log(`✅ Email ${job.id} terkirim: ${result.messageId}`);
});

worker.on('failed', (job, err) => {
  console.error(`❌ Email ${job.id} gagal: ${err.message}`);
});

console.log('Email worker ready...');
```

Auto-Retry Kalau SMTP Error

```
// Saat add job
await emailQueue.add('critical-email', payload, {
  attempts: 5,
  backoff: { type: 'exponential', delay: 1000 },
  removeOnComplete: false, // simpan history
});
```

---

2. PDF Generation (Puppeteer + Queue)

Generate PDF dari HTML itu CPU-intensive. Queue biar gak blocking server.

Setup

```
npm install puppeteer bullmq ioredis
```

Worker PDF

```
// pdf-worker.js
const { Worker } = require('bullmq');
const puppeteer = require('puppeteer');
const path = require('path');
const connection = require('./connection');

const worker = new Worker('pdf', async (job) => {
  await job.updateProgress(10);

  const { html, outputPath, options } = job.data;

  // Launch browser
  const browser = await puppeteer.launch({
    headless: 'new',
    args: ['--no-sandbox', '--disable-setuid-sandbox'],
  });

  await job.updateProgress(30);

  try {
    const page = await browser.newPage();

    // Set konten HTML
    await page.setContent(html, { waitUntil: 'networkidle0' });

    await job.updateProgress(60);

    // Generate PDF
    await page.pdf({
      path: outputPath,
      format: options?.format || 'A4',
      margin: { top: '2cm', bottom: '2cm' },
      printBackground: true,
    });

    await job.updateProgress(90);

    console.log(`PDF generated: ${outputPath}`);
    return { path: outputPath, size: require('fs').statSync(outputPath).size };
```

```
  } finally {
    await browser.close();
    await job.updateProgress(100);
  }
}, {
  connection,
  concurrency: 2, // 2 aja — puppeteer berat
  limiter: {
    max: 10,
    duration: 60000, // max 10 PDF per menit
  },
});

console.log('PDF worker ready...');
```

Producer PDF

```
// pdf-producer.js
const { Queue } = require('bullmq');
const connection = require('./connection');

const pdfQueue = new Queue('pdf', { connection });

async function generateInvoicePdf(invoice) {
  const html = getInvoiceHtml(invoice); // fungsi render HTML

  await pdfQueue.add('invoice-pdf', {
    html,
    outputPath: `/tmp/invoices/invoice-${invoice.id}.pdf`,
    options: { format: 'A4' },
  }, {
    attempts: 2,
  });
}
```

---

3. Image Processing (Sharp + Queue)

Resize image, bikin thumbnail, compress — semua berat dan blocking.

Setup

```
npm install sharp bullmq ioredis axios
```

Worker Image Processing

```
// image-worker.js
const { Worker } = require('bullmq');
const sharp = require('sharp');
const axios = require('axios');
const fs = require('fs/promises');
const path = require('path');
const connection = require('./connection');

const worker = new Worker('image', async (job) => {
  await job.updateProgress(10);

  const { imageUrl, uploadDir, filename } = job.data;
  const inputPath = path.join('/tmp', `input-${filename}`);
  const outputPath = path.join(uploadDir, filename);

  // Download image
  const response = await axios({
    url: imageUrl,
    responseType: 'arraybuffer',
  });
  await fs.writeFile(inputPath, response.data);

  await job.updateProgress(30);

  // Resize ke berbagai ukuran
  const sizes = [
    { width: 150, suffix: 'thumb' },   // thumbnail
    { width: 600, suffix: 'medium' },   // medium
    { width: 1200, suffix: 'large' },   // large
  ];

  await job.updateProgress(50);

  for (const size of sizes) {
    const outputFilename = filename.replace('.', `-${size.suffix}.`);
    const outputFile = path.join(uploadDir, outputFilename);

    await sharp(inputPath)
      .resize(size.width)
      .jpeg({ quality: 80 })
      .toFile(outputFile);

    console.log(`Resized: ${outputFilename}`);
  }
```

```
  // Cleanup temp
  await fs.unlink(inputPath);

  await job.updateProgress(100);

  return {
    original: outputPath,
    sizes: sizes.map(s => ({
      suffix: s.suffix,
      width: s.width,
    })),
  };
}, {
  connection,
  concurrency: 3,
});

console.log('Image worker ready...');
```

Producer Image

```
// image-producer.js
const { Queue } = require('bullmq');
const connection = require('./connection');

const imageQueue = new Queue('image', { connection });

async function processUserAvatar(imageUrl, userId) {
  const filename = `avatar-${userId}-${Date.now()}.jpg`;

  await imageQueue.add('avatar-processing', {
    imageUrl,
    uploadDir: `/uploads/avatars/${userId}`,
    filename,
  });
}
```

---

4. Notification Batching

Daripada kirim notifikasi 1 per 1, batch beberapa jadi 1. Hemat resource.

Queue per Notifikasi

```
// notif-producer.js
const { Queue } = require('bullmq');
const connection = require('./connection');

const notifQueue = new Queue('notifications', { connection });

async function sendNotification(userId, type, message) {
  await notifQueue.add('single-notif', {
    userId, type, message, timestamp: Date.now(),
  });
}
```

Worker dengan Batching Logic

```
// notif-worker.js — batch scheduler
const { Worker, Queue } = require('bullmq');
const connection = require('./connection');

const BATCH_SIZE = 10;
const BATCH_INTERVAL = 5000; // 5 detik

const worker = new Worker('notifications', async (job) => {
  // Worker per-job — gak di-batch disini
  // (langsung kirim notif individual)
}, { connection });

// Batching via cron job terpisah
const schedulerQueue = new Queue('notif-scheduler', { connection });

// Drain queue tiap 5 detik — kumpulin & batch
async function drainQueue() {
  const queue = new Queue('notifications', { connection });

  let batch = [];
  let job;

  // Ambil job dari queue sampai kosong atau batch penuh
  while (batch.length < BATCH_SIZE) {
    job = await queue.getActive(); // ambil job yang active
    // simplified — real flow lebih kompleks
    break;
  }

  if (batch.length > 0) {
```

```
    // Kirim batch notification (FCM, push, dll)
    await sendBatchNotification(batch);
  }
}

// Jalankan drain tiap 5 detik
setInterval(drainQueue, BATCH_INTERVAL);
```

Batching cocok buat: - Email marketing (kumpulin 100 dulu baru kirim) - Push notification (FCM support batch) - SMS gateway (biasanya rate-limited) - Analytics event

---

5. Bulk Data Export

Export ribuan record ke CSV/Excel lewat queue — user gak nunggu.

Queue Export

```
// export-producer.js
const { Queue } = require('bullmq');
const connection = require('./connection');

const exportQueue = new Queue('export', { connection });

async function requestExport(userId, filters, format = 'csv') {
  const job = await exportQueue.add('data-export', {
    userId,
    filters,
    format,
    requestedAt: new Date().toISOString(),
  });

  return {
    jobId: job.id,
    message: 'Export sedang diproses. Notifikasi akan dikirim setelah selesai.',
  };
}
```

Worker Export

```
// export-worker.js
const { Worker } = require('bullmq');
const { createObjectCsvWriter } = require('csv-writer');
const ExcelJS = require('exceljs');
```

```
const connection = require('./connection');

const worker = new Worker('export', async (job) => {
  await job.updateProgress(10);
  const { userId, filters, format } = job.data;

  // Ambil data dari database (simulasi)
  const records = await queryDatabase(filters);
  await job.updateProgress(40);

  const outputPath = `/tmp/exports/${userId}-${Date.now()}.${format}`;

  if (format === 'csv') {
    await exportToCsv(records, outputPath);
  } else if (format === 'xlsx') {
    await exportToExcel(records, outputPath);
  }

  await job.updateProgress(80);

  // Upload ke storage (S3 / lokal)
  const fileUrl = await uploadToStorage(outputPath);

  await job.updateProgress(100);

  return { fileUrl, recordCount: records.length, format };
}, { connection, concurrency: 2 });

async function exportToCsv(records, outputPath) {
  const writer = createObjectCsvWriter({
    path: outputPath,
    header: Object.keys(records[0] || {}).map(key => ({ id: key, title: key })),
  });
  await writer.writeRecords(records);
}

async function exportToExcel(records, outputPath) {
  const workbook = new ExcelJS.Workbook();
  const sheet = workbook.addWorksheet('Export');

  sheet.columns = Object.keys(records[0] || {}).map(key => ({
    header: key, key, width: 20,
  }));

  sheet.addRows(records);
  await workbook.xlsx.writeFile(outputPath);
```

```
}

// Helper — query database
async function queryDatabase(filters) {
  // Simulasi query
  return Array.from({ length: 100 }, (_, i) => ({
    id: i + 1,
    name: `User ${i + 1}`,
    email: `user${i + 1}@example.com`,
    createdAt: new Date().toISOString(),
  }));
}
```

Latihan

Latihan 1: Email Queue System

Buat queue `email` dengan 2 tipe job: `welcome` dan `reset-password`. Worker kirim email via Nodemailer (pake Mailtrap.io atau ethereal.email). Tiap job punya retry 3x. Log messageId ke console.

Latihan 2: PDF Invoice Generator

Buat queue `pdf-gen`. Producer terima data invoice (nomor, tanggal, items, total). Worker render HTML invoice → generate PDF pake Puppeteer. Simpan ke `/tmp/invoices/`. Return file path.

Latihan 3: Image Thumbnail Pipeline

Buat queue `image-pipeline`. Worker download image dari URL (pake axios), resize ke 3 ukuran (150px, 600px, 1200px) pake Sharp. Simpan dengan suffix `-thumb`, `-medium`, `-large`. Return daftar file output.

Latihan 4: Bulk CSV Export

Buat queue `bulk-export`. Worker generate CSV dari 1000 dummy records pake `csv-writer`. Update progress tiap 100 record. Setelah selesai, simpan path file di return value. Producer polling status sampai job completed.

04. Queue Production

Queue Dashboard UI (Bull Board)

Bull Board = dashboard GUI buat monitor queue BullMQ. Bisa lihat job, retry, pause, semuanya dari browser.

Setup Bull Board

```
npm install @bull-board/express bullmq ioredis express
```

Integrasi dengan Express

```
// server-dashboard.js
const express = require('express');
const { createBullBoard } = require('@bull-board/api');
const { BullMQAdapter } = require('@bull-board/api/bullMQAdapter');
const { ExpressAdapter } = require('@bull-board/express');
const { Queue } = require('bullmq');
const IORedis = require('ioredis');

const app = express();
const connection = new IORedis({ host: 'localhost', port: 6379 });

// Buat queue instances
const emailQueue = new Queue('email', { connection });
const pdfQueue = new Queue('pdf', { connection });
const imageQueue = new Queue('image', { connection });
const exportQueue = new Queue('export', { connection });

// Setup Bull Board
const serverAdapter = new ExpressAdapter();
serverAdapter.setBasePath('/admin/queues');

createBullBoard({
  queues: [
    new BullMQAdapter(emailQueue),
    new BullMQAdapter(pdfQueue),
    new BullMQAdapter(imageQueue),
    new BullMQAdapter(exportQueue),
  ],
  serverAdapter,
});

app.use('/admin/queues', serverAdapter.getRouter());
```

```
// API routes lain
app.get('/', (req, res) => {
  res.json({ message: 'Queue Dashboard at /admin/queues' });
});

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
  console.log('Dashboard: http://localhost:3000/admin/queues');
});
```

Fitur Bull Board

Fitur	Deskripsi
Queue List	Lihat semua queue dan statusnya
Job Overview	Jumlah waiting, active, completed, failed, delayed
Job Detail	Lihat data, stack trace, progress
Retry Job	Retry job yang failed — 1 klik
Add Job	Manual nambah job buat testing
Pause/Resume	Stop queue sementara
Clean	Hapus completed/failed jobs

Dashboard dengan Autentikasi

```
// Tambah basic auth
const basicAuth = require('express-basic-auth');

app.use('/admin/queues', basicAuth({
  users: { admin: process.env.DASHBOARD_PASSWORD || 'secret' },
  challenge: true,
}), serverAdapter.getRouter());
```

Error Handling & Dead Letter Queue

Dead Letter Queue (DLQ)

Job yang gagal total (udah max attempts) dikirim ke queue khusus — “dead letter”. Biar gak bercampur sama job normal.

```
// dlq-setup.js
const { Queue, Worker } = require('bullmq');
const connection = require('./connection');
```

```
// Queue utama
const emailQueue = new Queue('email', { connection });
// Dead letter queue
const deadLetterQueue = new Queue('email-dlq', { connection });

const worker = new Worker('email', async (job) => {
  const result = await sendEmail(job.data);
  return result;
}, { connection });

// Tangkap failed jobs — kirim ke DLQ
worker.on('failed', async (job, err) => {
  if (job.attemptsMade >= job.opts.attempts) {
    // Job udah max percobaan — kirim ke DLQ
    await deadLetterQueue.add(job.name, job.data, {
      ...job.opts,
      attempts: 1, // DLQ cukup 1x
      originalJobId: job.id,
      failedReason: err.message,
      failedAt: new Date().toISOString(),
    });
    console.log(`□ Job ${job.id} pindah ke DLQ: ${err.message}`);
  }
});
```

Worker DLQ — Investigasi Manual

```
// dlq-worker.js
const { Worker } = require('bullmq');
const connection = require('./connection');

const dlqWorker = new Worker('email-dlq', async (job) => {
  console.error(`[DLQ] Job asal: ${job.data.originalJobId}`);
  console.error(`[DLQ] Alasan: ${job.data.failedReason}`);
  console.error(`[DLQ] Data:`, job.data);

  // Kirim alert ke admin (email, Slack, dll)
  await sendAlertToAdmin({
    type: 'DLQ_JOB',
    queue: 'email',
    originalJobId: job.data.originalJobId,
    error: job.data.failedReason,
  });
```

```
  // Jangan retry — biar admin yang investigasi
  return { alerted: true };
}, { connection });

dlqWorker.on('completed', (job) => {
  console.log(`DLQ alert sent for job ${job.data.originalJobId}`);
});
```

Logging & Monitoring Error

```
// error-logging.js — integrasi dengan logging service
worker.on('failed', async (job, err) => {
  // Log ke file
  const logEntry = {
    timestamp: new Date().toISOString(),
    queue: 'email',
    jobId: job.id,
    jobName: job.name,
    attempts: job.attemptsMade,
    maxAttempts: job.opts.attempts,
    error: err.message,
    stack: err.stack,
    data: job.data,
  };

  await fs.appendFile('queue-errors.log', JSON.stringify(logEntry) + '\n');

  // Kirim ke Sentry / Datadog / etc.
  // Sentry.captureException(err, { extra: { jobId: job.id } });
});
```

Error Categories & Strategy

Error	Strategy	Retry?	DLQ?
Rate limited	Exponential backoff	Yes	No
SMTP timeout	Retry (mungkin sementara)	Yes (3x)	Jika masih gagal
Invalid email	Skip — discard job	No	Yes
Template error	Fix code — gak retry	No	Yes
DB connection	Retry + alert	Yes (5x)	Jika masih gagal

Worker Scaling

Multiple Workers (Horizontal)

Cara paling simpel: jalan banyak worker process.

```
# Start 3 worker instances
node email-worker.js &
node email-worker.js &
node email-worker.js &

# Atau pake PM2 cluster mode
pm2 start email-worker.js -i 4 --name email-worker
```

```
// pm2-ecosystem.config.js
module.exports = {
  apps: [{
    name: 'email-worker',
    script: 'email-worker.js',
    instances: 4, // 4 instance
    exec_mode: 'cluster',
    env: {
      NODE_ENV: 'production',
      REDIS_URL: 'redis://localhost:6379',
    },
  }, {
    name: 'pdf-worker',
    script: 'pdf-worker.js',
    instances: 2,
    exec_mode: 'cluster',
  }],
};
```

Concurrency Tuning

Gak semua queue sama — concurrency harus disesuaikan.

```
// Email worker — I/O heavy, bisa banyak concurrent
const emailWorker = new Worker('email', handler, {
  connection,
  concurrency: 20, // banyak karena I/O bound
});

// PDF worker — CPU heavy, concurrent sedikit
const pdfWorker = new Worker('pdf', handler, {
  connection,
  concurrency: 2, // CPU bound, 2 aja cukup
});
```

```
// Image worker — mixed, moderate
const imageWorker = new Worker('image', handler, {
  connection,
  concurrency: 4,
});
```

Aturan Concurrency

Tipe	Contoh	Concurrency Rekomendasi
I/O bound	Email, HTTP call, DB query	10-50
CPU bound	PDF, image resize, video	1-4
Mixed	Export, aggregation	4-10
External API	Rate-limited	Sesuai rate limit

Queue Prioritization

Job penting bisa didahulukan.

```
// Priority — nilai terkecil didahulukan
await emailQueue.add('verify-email', data, { priority: 1 });  // Tertinggi
await emailQueue.add('newsletter', data, { priority: 10 });   // Rendah
await emailQueue.add('promo', data, { priority: 5 });          // Medium
```

```
graph TD
    subgraph Priority Queue
        P1[Priority 1: Verify Email] --> Active
        P5[Priority 5: Promo] --> Active
        P10[Priority 10: Newsletter] --> Active
    end
    Active -->|I/O Available| W1[Worker 1]
    Active -->|I/O Available| W2[Worker 2]
    Active -->|I/O Available| W3[Worker 3]
```

Graceful Shutdown

Worker jalan terus. Waktu server mau mati (deploy, restart, crash), kita harus pastikan job yang lagi diproses gak hilang.

Problem: Kill Paksa

```
# □ Jangan — job aktif hilang
kill -9 <pid>
```

```
# Atau Ctrl+C paksa
```

Process mati mendadak → job jadi **stalled** (stuck di active state).

Solution: Graceful Shutdown

```
// graceful-shutdown.js
const { Worker, Queue } = require('bullmq');
const connection = require('./connection');

const worker = new Worker('email', async (job) => {
  // Proses email...
  await sendEmail(job.data);
}, { connection });

// Handler graceful shutdown
async function shutdown(signal) {
  console.log(`\n${signal} received — shutting down gracefully...`);

  // 1. Stop worker — gak ambil job baru
  await worker.close({ force: false });
  // force: false — biarin job yg lagi jalan selesai

  console.log('Worker closed. Remaining jobs will be picked up later.');

  // 2. Tutup koneksi Redis
  await connection.quit();

  console.log('Redis connection closed.');
  process.exit(0);
}

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));

console.log('Worker running. Press Ctrl+C to shutdown gracefully.');
```

Stalled Job Recovery

BullMQ otomatis detek stalled job (di active > `stalledInterval`) dan mindahin balik ke waiting.

```
const worker = new Worker('email', handler, {
  connection,
  stalledInterval: 30000,     // cek tiap 30 detik
```

```
  maxStalledCount: 2,         // max 2x stalled sebelum failed
});
```

Docker Graceful Shutdown

```
# Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .
CMD ["node", "worker.js"]
# Docker akan kirim SIGTERM — worker.js handle itu
```

```
# docker-compose.yml
version: '3.8'
services:
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  email-worker:
    build: .
    depends_on: [redis]
    environment:
      - REDIS_URL=redis://redis:6379
    stop_signal: SIGTERM
    stop_grace_period: 60s # kasih waktu 60s buat selesaiin job
```

Complete Production Worker Template

```
// production-worker.js
const { Worker, Queue } = require('bullmq');
const IORedis = require('ioredis');

const connection = new IORedis(process.env.REDIS_URL || 'redis://localhost:6379'
const QUEUE_NAME = process.env.QUEUE_NAME || 'default';

async function processJob(job) {
  // Implementasi proses job
  console.log(`[${new Date().toISOString()}] Processing ${job.id}`);
  return { processed: true };
}

const worker = new Worker(QUEUE_NAME, processJob, {
  connection,
```

```
  concurrency: parseInt(process.env.CONCURRENCY || '5'),
  stalledInterval: 30000,
  maxStalledCount: 2,
});

worker.on('completed', (job) => {
  console.log(`□ ${job.id} done`);
});

worker.on('failed', (job, err) => {
  console.error(`□ ${job.id} failed:`, err.message);
});

// Graceful shutdown
async function shutdown(signal) {
  console.log(`\n${signal} received. Closing worker...`);
  await worker.close({ force: false });
  await connection.quit();
  process.exit(0);
}

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));

console.log(`Worker ${QUEUE_NAME} ready (concurrency: ${worker.opts.concurrency}`
```

Cron Job vs Queue Decision Guide

Kapan pake cron job biasa (node-cron, setInterval) dan kapan pake queue + scheduler?

Cron Job

```
// Cron job sederhana — cocok buat task ringan
const cron = require('node-cron');

cron.schedule('0 8 * * *', () => {
  console.log('Kirim laporan harian');
  generateDailyReport();
});
```

Cocok buat	Gak cocok buat
Task ringan < 1 detik	Task berat / lama
Gak perlu retry	Butuh retry logic
Hanya 1 process	Butuh scaling
Gak perlu monitoring	Butuh dashboard
Gak masalah kalau gagal	Critical task

Queue (BullMQ Scheduler)

```
// Queue scheduler — cocok buat task production
const { Queue } = require('bullmq');

const reportQueue = new Queue('reports', { connection });

await reportQueue.add('daily-report', data, {
  repeat: { pattern: '0 8 * * *' },
  attempts: 3,
  backoff: { type: 'exponential', delay: 5000 },
});
```

Cocok buat	Fitur tambahan
Task berat / lama	Retry otomatis
Scaling horizontal	Multiple workers
Critical path	DLQ + alerting
Butuh monitoring	Bull Board dashboard
Task per-user / banyak	Concurrency control

Decision Matrix

```
Apakah task bisa gagal dan perlu diulang?
├── Ya → Queue (retry + DLQ)
└── Tidak → Cron job

Apakah task perlu scale saat ramai?
├── Ya → Queue (concurrent workers)
└── Tidak → Cron job

Apakah task critical / financial?
├── Ya → Queue (monitoring + audit trail)
└── Tidak → Bisa cron job aja

Apakah task per-user (masing-masing beda data)?
├── Ya → Queue (1 job per user)
```

```
└── Tidak → Cron job (1 task buat semua)

Apakah server sering restart / deploy?
├── Ya → Queue (recover stalled jobs)
└── Tidak → Cron job
```

Hybrid Approach

Kadang perlu kombinasi:

```
// Cron job buat trigger queue
cron.schedule('0 8 * * *', () => {
  // Cron cuma trigger — kerja berat di queue worker
  reportQueue.add('daily-report', {
    date: new Date().toISOString().split('T')[0],
  });
});
```

Latihan

Latihan 1: Bull Board Setup

Setup Express server dengan Bull Board. Daftarin 3 queue: `email`, `pdf`, `image`. Tambahin basic auth. Jalankan dan pastikan dashboard bisa diakses di `/admin/queues`.

Latihan 2: Dead Letter Queue

Buat queue `orders` dengan worker yang sengaja throw error. Konfigurasi `attempts: 2`. Di event `failed`, kirim job ke `orders-dlq` kalau udah max attempts. Buat DLQ worker yang log detail error dan marked as alerted.

Latihan 3: Graceful Shutdown

Buat worker yang proses job dengan delay 3 detik (simulasi). Handle SIGTERM: worker.close(), biarin job selesai, baru exit. Test dengan Ctrl+C — pastikan job terakhir tetep completed bukan failed.

Latihan 4: Scaling Decision

Buat laporan analisis: dari 5 use case berikut, tentuin mana yang pake cron job dan mana yang pake queue. Beri alasan: 1. Bersihin log file setiap jam 2. Kirim email notifikasi pas user register 3. Generate

thumbnaily pas user upload foto 4. Backup database setiap tengah malam 5. Proses refund order yang dibatalkan